SYSTEM DESIGN

Fundamental Areas to Master

A practical roadmap for learning the concepts, trade-offs, and principles behind scalable, reliable backend and data-intensive systems.

15 core areas	A structured map of the knowledge to build
4 learning stages	A suggested order for learning the fundamentals
5 design questions	A repeatable checklist for every architecture

The goal is not to memorize hundreds of designs. It is to learn a compact set of fundamentals and apply them to different problems.

Part 1 — The 15 Fundamental Areas

Use this as a checklist. You do not need equal depth in every topic; build strong fundamentals first, then go deeper in areas relevant to your work and target roles.

1. Networking and communication

- HTTP/HTTPS, TCP/IP, DNS
- REST APIs, gRPC, WebSockets
- Load balancers and reverse proxies
- Latency, bandwidth, connection pooling
- Timeouts, retries, and idempotency

2. Databases and data modeling

- Relational vs. NoSQL databases
- Schema design, normalization, and denormalization
- Indexes and query optimization
- Transactions, ACID, and isolation levels
- Partitioning, replication, and read replicas
- SQL, key-value, document, graph, and wide-column stores

3. Distributed systems fundamentals

- Horizontal vs. vertical scaling
- Replication and partitioning
- Consistency models and quorum
- CAP theorem and PACELC
- Distributed coordination and consensus
- Leader election, split brain, and network partitions
- Distributed transactions and eventual consistency

4. Caching

- Cache-aside, read-through, and write-through
- Redis and in-memory caches
- TTL and eviction policies
- Cache invalidation and stale data
- Hot keys and cache stampedes
- CDN and browser caching

5. Asynchronous processing and messaging

- Message queues vs. event streams
- Kafka, RabbitMQ, and pub/sub
- Producers, consumers, and consumer groups
- At-most-once, at-least-once, and effectively-once processing
- Ordering, retries, and dead-letter queues
- Backpressure and event-driven architecture

6. Application and service architecture

- Monolith vs. microservices
- API Gateway and service discovery
- Service boundaries and domain modeling
- Synchronous vs. asynchronous communication
- Stateless services and session management
- Service decomposition and dependency management

7. Scalability, availability, and reliability

- Horizontal scaling and autoscaling
- Redundancy and fault tolerance
- High availability and disaster recovery
- Failover, circuit breakers, and bulkheads
- Graceful degradation
- SLI, SLO, SLA, RTO, and RPO

8. Performance and capacity planning

- Requests per second and concurrent users
- Read/write ratios
- Storage and bandwidth estimates
- Latency budgets and throughput
- Load testing and bottleneck analysis
- Capacity planning and performance trade-offs

9. Security

- Authentication vs. authorization
- OAuth 2.0, OIDC, and JWT
- TLS and encryption at rest and in transit
- Secrets management and key rotation
- Rate limiting and abuse prevention
- Least privilege and audit logging

10. Observability and operations

- Metrics, logs, and distributed traces
- Dashboards and alerting
- Health checks and readiness probes
- Incident response and root-cause analysis
- Deployment strategies and rollback
- Configuration and feature flags

11. Data-intensive system design

- Batch vs. stream processing
- Data lakes, warehouses, and lakehouses
- Partitioning, file formats, and data locality
- ETL/ELT and change data capture (CDC)
- Stream processing, event time, and late events
- Data consistency, replay, and deduplication

12. Architectural trade-offs and design principles

- Consistency vs. availability
- Latency vs. throughput
- Cost vs. reliability
- Simplicity vs. flexibility
- Strong vs. eventual consistency
- Single points of failure and failure isolation
- Build vs. buy; premature optimization vs. justified scaling

13. Common architectural patterns

- CQRS and Event Sourcing
- Saga pattern and Outbox pattern
- Strangler and bulkhead patterns
- Sharding, fan-out, and aggregation
- Leader-follower and producer-consumer patterns

14. Core software design principles

- Separation of concerns and modularity
- SOLID principles and dependency inversion
- Composition over inheritance
- Domain-driven design fundamentals
- Clear ownership of data and responsibilities

15. System design methodology

- Clarify functional and non-functional requirements
- Estimate traffic, storage, and throughput
- Define APIs and data models
- Draw the high-level architecture
- Identify bottlenecks and failure scenarios
- Explain trade-offs and evolve the design incrementally

Part 2 — A Practical Learning Order

Stage 1 — Foundation

Focus: Networking, APIs, databases, indexes, transactions, and caching.

Outcome: Understand how a single backend application works and where its data and latency costs come from.

Stage 2 — Scaling and distributed systems

Focus: Load balancing, replication, partitioning, consistency, messaging, and fault tolerance.

Outcome: Learn how multiple services and machines coordinate, scale, and respond to failures.

Stage 3 — Production architecture

Focus: Security, observability, deployment, disaster recovery, and capacity planning.

Outcome: Learn how to operate a reliable production system and diagnose problems.

Stage 4 — Specialization and practice

Focus: Big data architecture, architectural patterns, trade-off analysis, and system design exercises.

Outcome: Apply the fundamentals to progressively more complex systems.

Part 3 — Four Questions for Every Concept

■ What problem does it solve?

■ How does it work?

■ What are its limitations or failure modes?

■ When would you choose it over an alternative?

Example: Redis caching
Do not stop at knowing what Redis is. Understand why caching helps, how invalidation works, how a cache stampede occurs, what stale data means, and when the added complexity is not justified.

Part 4 — The Five Questions Behind Every Design

1. How does it scale? What grows first: traffic, data, CPU, memory, or network usage?

2. How does it handle failures? What happens when a server, database, broker, or network link fails?

3. How does it maintain data correctness? What consistency guarantees, ordering, transactions, or deduplication are required?

4. Where are its bottlenecks? Which component limits throughput or creates unacceptable latency?

5. What trade-offs justify the architecture? Why is this design better for the requirements than a simpler or different alternative?

A Final Recommendation

For backend, distributed systems, and big data preparation, emphasize distributed systems fundamentals, database internals, messaging, performance, and data-intensive architecture. Spend less time memorizing product names and more time understanding mechanisms and trade-offs.

Central idea: System design is not about knowing the largest number of technologies or patterns. It is about using a compact set of fundamentals to make sound architectural decisions under different constraints.